

Guía de Estudio: JavaScript

Fundamentos, Operadores Lógicos y Escalabilidad de Proyectos

Esta guía de estudio está diseñada para ayudarte a repasar a nivel de código puro. Es ideal para prepararte antes de un examen o evaluación técnica, permitiéndote comprender la sintaxis exacta (símbolos lógicos) y visualizar cómo agregar nuevas características a un proyecto existente paso a paso.

1. Operadores Lógicos y Sintaxis (Repaso)

En JavaScript, las decisiones del flujo de la aplicación se toman usando operadores. Aquí te explicamos los más importantes que debes dominar:

A. El Operador Ternario (? :) - El "If/Else" de una sola línea

Este operador es muy común para la lectura de bases de datos o almacenamiento local. En archivos de manejo de datos, es probable ver esta estructura:

```
const data = localStorage.getItem(STORAGE_KEY);  
return data ? JSON.parse(data) : [];
```

¿Qué significa? Es una evaluación condicional directa. La sintaxis es: `condicion ? (qué hacer si es verdad) : (qué hacer si es falso)`.

NOTA DE ESTUDIO: ¿POR QUÉ USARLO?

Se utiliza por eficiencia y legibilidad. Primero busca la llave en el LocalStorage. La condición evalúa si `data` tiene contenido. Si es verdadero (`?`), convierte el texto a un arreglo usando `JSON.parse(data)`. Si es falso o está vacío (`:`), devuelve un arreglo vacío `[]` para evitar que la aplicación colapse al intentar leer datos inexistentes.

B. El Operador AND (&& - "Y")

El operador `&&` exige que todas las condiciones sean verdaderas al mismo tiempo. Es fundamental en módulos de autenticación y validaciones estrictas.

```
// Ejemplo de validación múltiple  
if (usuarioIngresado === emailGuardado && passwordIngresada === passwordGuardada) {  
  console.log("¡Bienvenido, puedes entrar!");  
}
```

NOTA DE ESTUDIO: COMPORTAMIENTO ESTRICTO

Si el email es correcto, pero la contraseña no (o viceversa), todo el bloque se evalúa como `false` y se bloquea el acceso. Ambas partes unidas por el `&&` deben cumplirse obligatoriamente.

C. El Operador OR (|| - "O")

El operador `||` se usa cuando necesitas que al menos una condición sea verdadera, o frecuentemente en JavaScript, para asignar **valores por defecto** (Default values).

```
// Ejemplo de asignación segura de datos
const nombreEstudiante = datos.nombre || "Usuario Anónimo";
```

NOTA DE ESTUDIO: RESPALDO DE VARIABLES

Si el objeto `datos` tiene un nombre válido (ej. 'Carlos'), la variable guarda 'Carlos'. Pero si el nombre viene vacío, nulo o `undefined`, el operador `||` actúa como un respaldo de seguridad y le asigna el valor 'Usuario Anónimo'.

2. Escalabilidad: ¿Cómo añadir una nueva característica?

Un excelente ejercicio de estudio es simular la adición de una nueva funcionalidad. Supongamos que necesitas que los exámenes tengan un nivel de dificultad (Fácil, Medio, Difícil). Entender este flujo demuestra que comprendes la arquitectura de datos del proyecto:

Paso 1: Modificar el Modelo de Datos

Debes ubicar la plantilla o clase que define la estructura del objeto. Lo primero es añadir la nueva propiedad a ese molde base:

```
const ExamenTemplate = {
  id: null,
  titulo: '',
  tiempo: 0,
  // ... otros datos ...
  dificultad: 'Fácil' // <-- 1. Se define la nueva propiedad por defecto
};
```

Paso 2: Actualizar la Interfaz (HTML)

Luego, debes ir a la vista donde el usuario ingresa los datos e incluir un nuevo campo, como un menú desplegable (`<select>`).

```
<div class="form-group">
  <label>Dificultad del Examen:</label>
```

```

    <select id="input-dificultad">
      <option value="Fácil">Fácil</option>
      <option value="Medio">Medio</option>
      <option value="Difícil">Difícil</option>
    </select>
  </div>

```

Paso 3: Capturar y Procesar en JavaScript

En el controlador, debes capturar el valor que el usuario seleccionó y agregarlo al momento de construir el nuevo objeto antes de guardarlo.

```

// Capturar el valor ingresado en el DOM
const dificultadSeleccionada = document.getElementById('input-dificultad').value;

// Armar el objeto integrando el nuevo dato
const nuevoExamen = {
  id: Date.now(),
  titulo: document.getElementById('input-titulo').value,
  dificultad: dificultadSeleccionada // <-- 2. Se almacena la propiedad
};

// Guardar en persistencia (ej. LocalStorage)
ExamenStorage.guardarExamen(nuevoExamen);

```

Paso 4: Renderizar la Información

Por último, utilizarás Plantillas Literales (Template Literals) con los símbolos ` (backticks) y `\${}` para inyectar esta nueva variable visualmente en la interfaz de lectura.

```

// Inyectando la nueva variable dinámicamente en el renderizado
const tarjetaHTML = `
  <div class="tarjeta-examen">
    <h4>\${examen.titulo}</h4>
    <p>Tiempo: \${examen.tiempo} minutos</p>
    <p>Nivel: <strong>\${examen.dificultad}</strong></p> <!-- 3. Se muestra al usuario
  -->
  </div>
`;

```

TÉCNICA DE ESTUDIO: ANÁLISIS DE CÓDIGO

Cuando te enfrentes a bloques de código complejos durante tu estudio, utiliza este método de 3 pasos para interiorizar su funcionamiento:

- **1. Identifica el método:** Reconoce qué herramienta o función de JavaScript estás utilizando (ej. un `.map()`, un `.filter()` o un operador ternario).
- **2. Entiende la mecánica:** Describe qué hace internamente la computadora (ej. "Itera sobre el arreglo y me devuelve un nuevo arreglo" o "evalúa si esta variable existe").
- **3. Justifica la decisión:** Piensa por qué se usó esa herramienta en lugar de otra (ej. "Hace el código más limpio, es más rápido o evita errores de ejecución").